

CHAP 7 - TIMERS
Informatique Embarquée
S2 2020- 2021
GEII - Toulouse III

A. Nketsa

Les Timers

1- Définition

Un timer est un compteur et/ou décompteur programmable matériel (physique) intégré ou externe commandé et contrôlé par un microcontrôleur.

Le compteur programmable est exactement comme le prédiviseur que nous avons utilisé en électronique numérique.

2- Objectif

L'objectif d'un timer dans un système à base de microcontrôleur

est de décharger le microcontrôleur des tâches d'attentes passives sans exploitation de ses capacités de calculs.

Exemples :

a) Comptage du temps

Pour compter le temps sans timer, on doit exploiter le temps d'exécution des instructions (en les accumulant) pour générer un temps. Cela revient à faire une boucle dont on calcule la durée d'un passage dans la boucle et à multiplier cette durée par le nombre de passages pour obtenir la durée de la tempo. Cela revient à la vue externe et l'organigramme ci-contre.

Soit d1 (resp d2) le temps d'exécution de l'instruction de décrémentation (resp de test).

On peut évaluer la durée de la fonction tempo en écrivant la relation

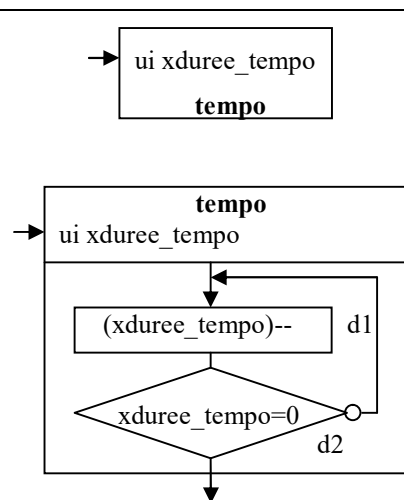
$$\text{Duree_tempo} = (d1 + d2) * \text{xduree_tempo}$$

Remarque :

Si xduree_tempo vaut 0 alors on prend dans la formule xduree_tempo = 65536.

Conclusion :

Le microcontrôleur perd son temps à compter le temps et n'exploite pas sa puissance de calcul pour d'autres tâches.



b) Attente d'un événement

Un événement est un changement d'état. Par exemple un front montant est un événement.

Front montant :

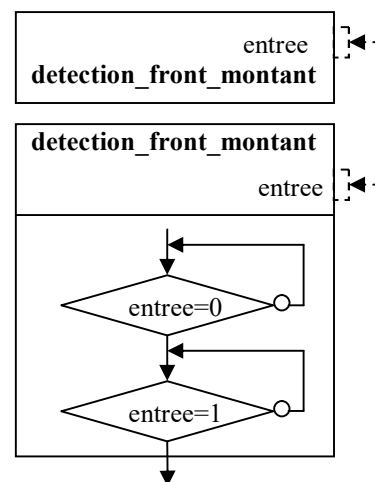
La détection d'un front montant par exemple consiste à attendre dans l'ordre:

- le niveau bas
- puis le niveau haut

Voir la vue externe et l'organigramme ci-contre

Conclusion :

Le microcontrôleur perd son temps à attendre les niveaux requis. Il ne peut plus rien faire d'autre.



3- Modes de fonctionnement

Un timer peut donc :

- compter le temps dans ce cas on l'appelle **timer**
la base de temps est alors l'horloge du microcontrôleur
- détecter les fronts sur une entrée et même les compter dans ce cas on l'appelle **compteur d'événements**

l'entrée est une broche dédiée du microcontrôleur.

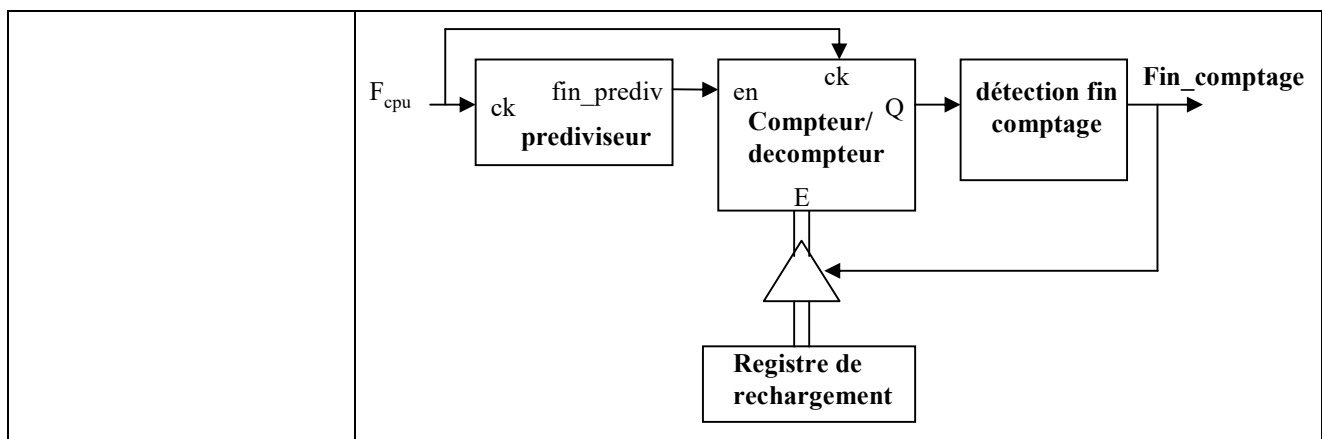
Il est évident que si cette entrée est connectée à une horloge (signal périodique), on peut aussi compter le temps.

4- Schéma de principe

Le schéma de principe est semblable à celui que nous avons étudié en électronique numérique concernant le PWM.

4-1 Mode Timer

Dans le mode Timer, on a une mise en cascade d'un prédiviseur et d'un compteur/décompteur avec la génération du signal, fin_comptage. (Exactement comme en ER FPGA- VHDL)



- Le prédiviseur est un décompteur dont la fin de décomptage permet de commander le compteur/décompteur qui est en cascade.
- Le compteur est un compteur/décompteur binaire de n bits.
 $\text{valeur_min} = 0x0000$ $\text{valeur_max} = 2^n - 1$
- Le registre de rechargement
- Le bloc, détection_fin_comptage, permet de générer le signal, fin_comptage dans les cas suivants :

Compteur/décompteur en compteur

fin_comptage passe à 1 quand le compteur passe de valeur_max à 0.

Compteur/décompteur en décompteur

fin_comptage passe à 1 quand le décompteur passe de 0 à valeur_max.

- Durée générée pour rendre fin_comptage actif = $T_{\text{cpu}} * \text{modulo_prediviseur} * \text{nbre_impulsions_compteur}$

4-2 Mode Compteur d'événements

Nous traiterons un cas particulier simple

Remarque importante :

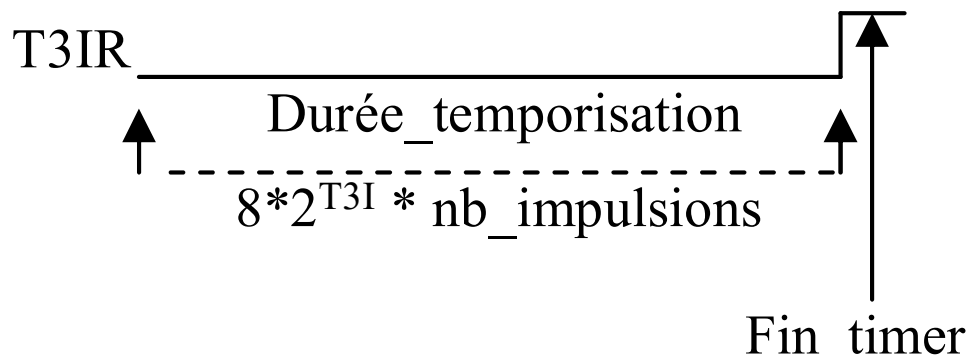
Dans cette configuration, le groupe GPT1 permet de réaliser :

☞ une temporisation de moyenne durée variant :

de $8 * T_{cpu}$ avec $T3I = 0$
à $8 * 2^{T3I} * 2^{16} * T_{cpu}$ avec $T3I = 7$

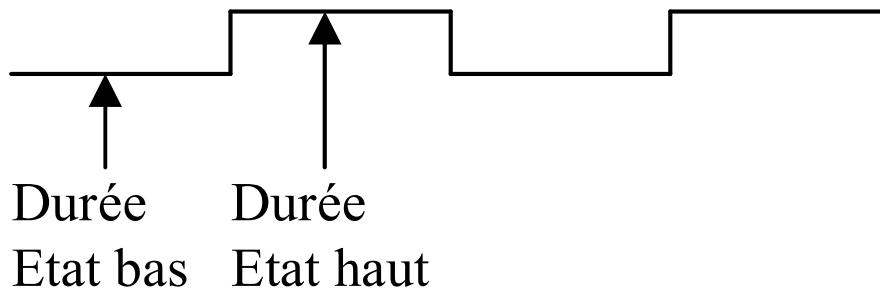
Exemple $F_{cpu} = 20\text{Mhz}$ $\Rightarrow T_{cpu} = 50\text{ns}$

on obtient une durée de 400ns à environ 3355443200ns $\approx 33,5\text{s}$



☞ la génération d'un signal carré sur T3OTL qui peut être recopié sur la broche d'entrée-sortie P3.3

On peut générer un signal carré périodique



$$durée_Etat_bas = duree_etat_haut = Période/2$$

b) Programmation

Pour programmer les Timers de GPT1, nous allons utiliser les registres de contrôle (T3CON et T2CON) et les registres de comptage-décomptage (T3 et T2)

b-1) Les registres de contrôle

Puisque nous avons réduit les modes de fonctionnement, nous allons simplifier la programmation.

Le registre T3CON pour le timer T3

Le registre de contrôle T3CON est organisé en champs de la façon suivante :

T3CON

Numéro bit	15 à 11					10	9	8	7	6	5	4	3	2	1	0
Nom signal						T3OTL	T3OE	T3UDE	T3UD	T3R	T3M			T3I		
Valeur imposée	0	0	0	0	0			0	1		0	0	0			

Les bits **T3OTL**, **T3OE**, **T3UD** et **T3R** sont connus par le compilateur du langage C du C167.

T3M permet de choisir le mode de fonctionnement du Timer T3.

Nous allons le fixer à **000** pour indiquer que T3 fonctionne en mode Timer (comptage du temps)

T3I permet de définir le modulo du prédiviseur lorsque le timer T3 fonctionne en mode Timer.

Ce modulo vaut $8 * 2^{T3I}$ T3I est sur 3 bits et peut donc prendre les valeurs de 0 à 7.

Ce qui conduit à des prédiviseurs de $8 * 2^0 = 8$ (T3I=0) à $8 * 2^7 = 1024$ (T3I=7)

T3I n'est pas connu par le compilateur, il faut trouver un mécanisme pour écrire cette valeur à la bonne position dans le registre T3CON. Pour cela nous allons procéder par madquage.

T3UDE ne sera pas utilisé. Nous le fixons à 0

T3OE est l'autorisation de la copie de T3OTL sur la broche d'entrée-sortie P3.3.

Nous le fixons à 1 pour permettre la génération de signal carré sur P3.3

T3OTL est le signal qui change d'état à chaque fin de comptage ou décomptage. Nous l'initialiserons à 0.

T3UD permet de choisir entre le comptage (T3UD = 0) et le décomptage (T3UD=1). Nous le fixerons à 1

T3R permet d'autoriser (T3R=1) ou d'arrêter (T3R=0) le comptage/décomptage.

A l'initialisation, T3CON = 0x0000 $\Rightarrow$ tous les bits et les champs sont à 0

Le registre T2CON pour le timer T2 (utilisé en registre de rechargement)

Le registre de contrôle T2CON est organisé en champs de la façon suivante :

T2CON

Numéro bit	15 à 9							8	7	6	5	4	3	2	1	0
Nom signal								T2UDE	T2UD	T2R	T2M			T2I		
Valeur imposée	0	0	0	0	0	0	0	0	0	0	1	0	0	1	1	1

Les bits **T2UD** et **T2R** sont connus par le compilateur du langage C du C167.

T2M permet de choisir le mode de fonctionnement du Timer T3. Nous allons le fixer à **100** pour indiquer que T2 fonctionne en mode registre de rechargement de T3

T2I permet de définir quand le rechargement de T3 doit se faire.

Comme nous voulons générer un signal carré sur P3.3, nous allons considérer que le rechargement se fera à toutes les fins de comptage ou décomptage c'est-à-dire sur les fronts montants et descendants de T3OTL. Pour cela, on doit programmer T2I = 111

T2UDE, **T2UD** et **T2R** ne seront pas utilisés. Nous les fixons à 0

Par conséquent la valeur de **T2CON** est fixe = **0000000000100111b = 0x027**

b-2) Les registres de comptage-décomptage

Les registres de comptage-décomptage sont des compteurs-décompteurs programmables dont on peut définir la valeur initiale:

- soit en écrivant directement dans le registre
- soit par le registre de rechargement automatique

Le seul registre de comptage décomptage pour le groupe GPT1 dans notre configuration est **T3**.

Rappel :

La fin du comptage est indiquée par T3IR =1

lorsque le compteur T3 passe de valeur_max à 0

La fin du décomptage est indiquée par T3IR =1

lorsque le décompteur T3 passe de 0 à valeur_max (0xFFFF)

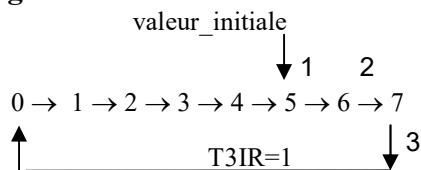
Nous allons considérer un exemple simple puis nous allons généraliser

La question est quelle valeur initiale faut-il programmer dans T3 pour qu'il signale la fin de comptage-décomptage de ces x impulsions par la mise à 1 T3IR

On sait que ce compteur-decompteur indique par $T3IR = 1$

- la fin du comptage lors du passage de 7 à 0
- ou la fin du décomptage lors du passage de 0 à 7

Comptage



Comme nous travaillons en binaire sur 3bits, on constate que $5 = -3$

Si le compteur-décompteur fonctionne *en compteur*, et on veut détecter le comptage de x impulsions,
il faut charger comme valeur initiale $-x$ soit $2^n - x$
 n étant le nombre de bits de compteur-décompteur.

valeur_initiale
 2 1
 0 ← 1 ← 2 ← 3 ← 4 ← 5 ← 6 ← 7
 3
 T3IR=1

On constate que $2 = 3-1$

Si le compteur-décompteur fonctionne en *décompteur*, et on veut détecter le décomptage de x impulsions, il faut charger comme valeur initiale $x-1$

La détection de fin de comptage ou décomptage se fait par

- ### Important:

Il est de la responsabilité du programmeur de remettre ce bit à 0 pour permettre la prochaine détection de fin de comptage-décomptage.

b-4) Conclusion

Pour programmer les timers de groupe **GPT1**, on a deux étapes

- la préparation des valeurs d'initialisation à programmer dans les registres, **T3CON, T2CON, T2 et T3**
- la programmation effective qui comporte 2 parties :
 - ☞ les fonctions d'initialisation
 - ☞ l'exploitation du Timer dans le programme principal

a) Préparation des valeurs d'initialisation.

On calcule les paramètres à programmer dans le timer en particulier le nombre d'impulsions permettant de déterminer la valeur à programmer dans le registre de comptage-décomptage.

Soit Tp_duree , la durée d'une temporisation ou de la demi-période d'un signal carré que l'on veut faire générer par les timers de GPT1

Soit $nb_impulsions$, le nombre d'impulsions correspondant à cette durée, Tp_duree .

On a la relation $Tp_duree = Tcpu * modulo_prediviseur * nb_impulsions$

$Tcpu = 50ns$ pour $fcpu = 20Mhz$

$Modulo\ prédiviseur = 8 * 2^{T3I}$

$$Tp_duree (ns) = 50ns * 8 * 2^{T3I} * nb_impulsions$$

On doit donc calculer $T3I$ et $nb_impulsions$ à partir de la relation

$$2^{T3I} * nb_impulsions = Tp_duree(ns)/400$$

On doit donc choisir le couple **T3I et nb_impulsions** sachant que :

☞ $T3I \leq 7$

☞ et $nb_impulsions \leq 65536$
(attention 65536 conduit à charger 0)

a-1) Pour cela, nous conseillons de dresser une table.

Pour chaque valeur de $T3I$, on calcule $nb_impulsions$

a-2) On identifie les couples $T3I$ et $nb_impulsions$ tel que $nb_impulsions \leq 65536$

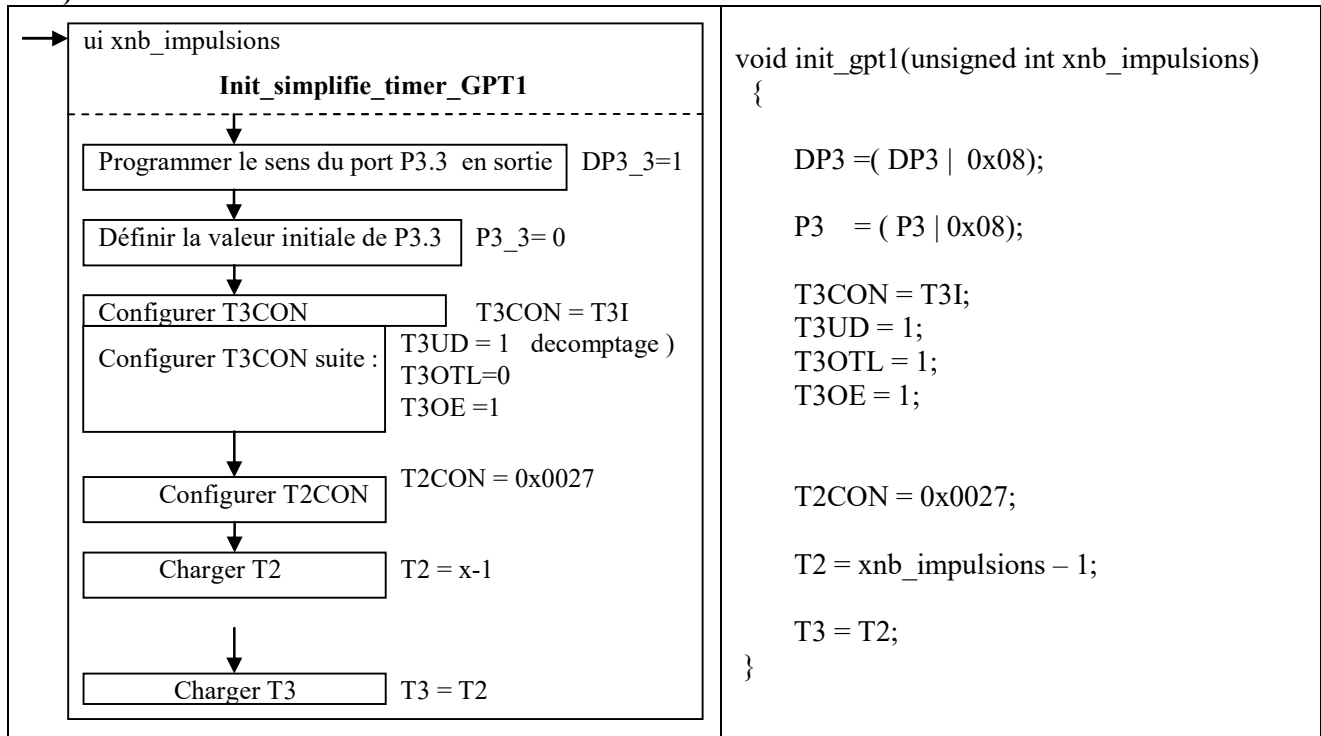
a-3) On choisit parmi les couples identifiés celui qui possède le plus petit T3I

Cette valeur de $T3I$ conduit à la plus grande précision sur la durée.

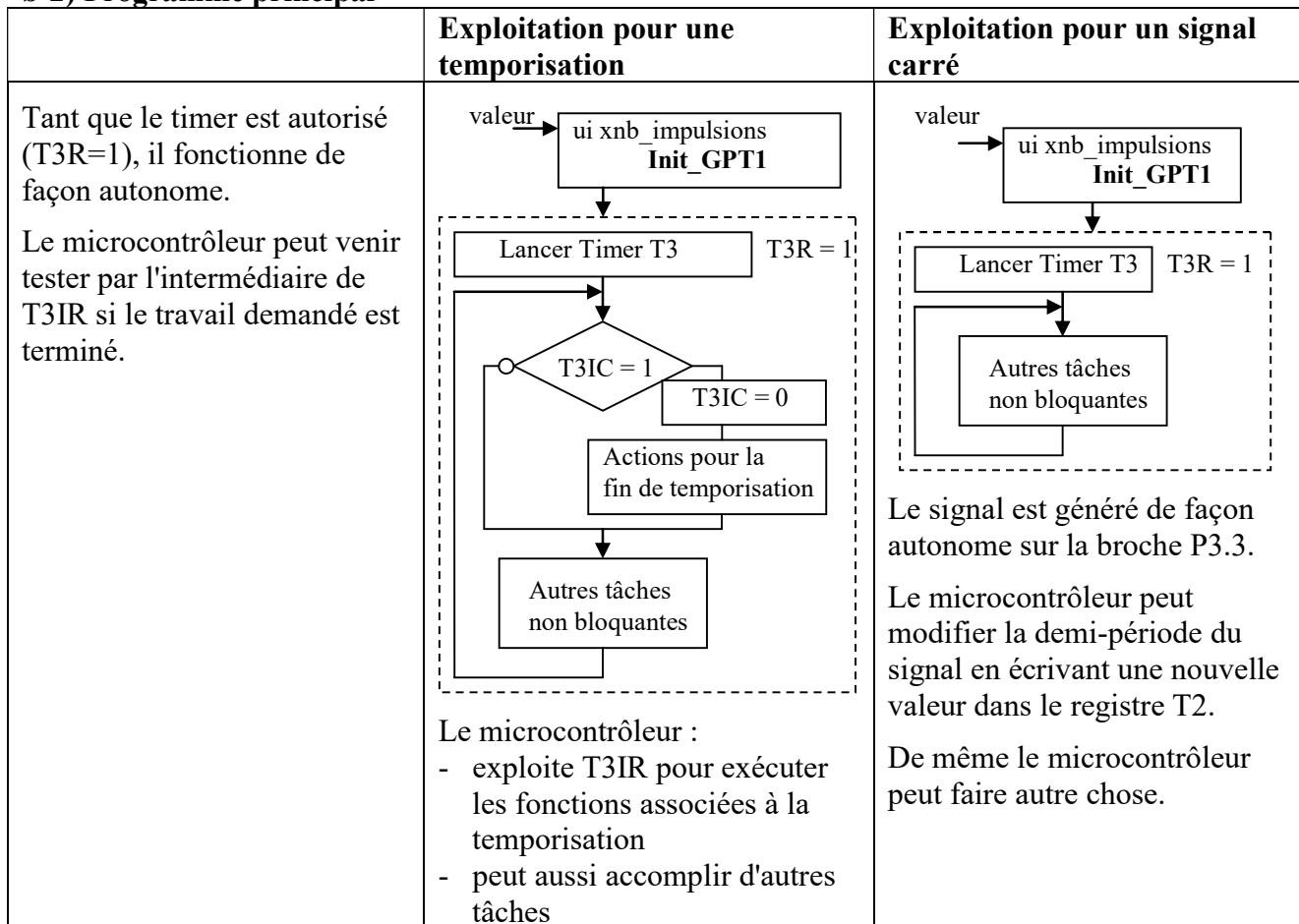
T3I	nb impulsions
0	
1	
2	
3	
4	
5	
6	
7	

b) Organigramme en utilisant les valeurs calculées en a)

b-1) Initialisation GPT1



b-2) Programme principal



Exemples

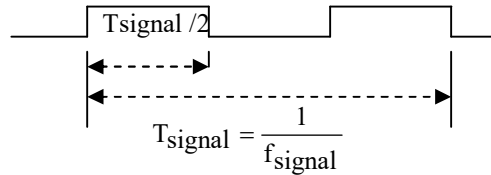
a) Générer un signal carré de fréquence **f_{signal}** (en Hz) sur la broche P3.3

On admet que f_{cpu} = 20Mhz

Méthode

Calcul des valeurs initiales

Signal à générer :



On doit donc programmer le timer T3 pour la génération d'un signal carré de demi-période = $T_{\text{signal}}/2$

Il suffit de calculer le nombre d'impulsions nécessaires en utilisant la formule :

$$2^{T3I} * \text{nb_impulsions} = (T_{\text{signal}}(\text{ns})/2)/400$$

$$(T_{\text{signal}}(\text{ns})/2)/400 = T_{\text{signal}}(\text{ns})/800 = (T_{\text{signal}}(\text{s}) * 10^9)/800 = T_{\text{signal}}(\text{s}) * 125 * 10^4$$

$$\text{ou encore } 2^{T3I} * \text{nb_impulsions} = (125 * 10^4)/(f_{\text{signal}}(\text{Hz}))$$

On en déduit T3I et nb_impulsions

Application numérique: f_{signal} = 1Khz

$$2^{T3I} * \text{nb_impulsions} = (125 * 10000)/1000 = 1250$$

Table

T3I	0	1	2	3	4	5	6	7
Nb_impulsions	1250	625	312,5	156,25	78,125	39,0625		

Il faut privilégier nb_impulsions entier.

Donc nous allons choisir le couple T3I = 0 nb_impulsions = 1250

La programmation revient à appliquer ces valeurs au code source présenté précédemment.

Programme

Initialisations

```
void init_gpt1(unsigned int xnb_impulsions)
{
    DP3 = ( DP3 | 0x08);

    P3  = ( P3 | 0x08);

    T3CON = T3I;
    T3UD = 1;
    T3OTL = 1;
    T3OE = 1;

    T2CON = 0x0027;

    T2 = xnb_impulsions - 1;

    T3 = T2;
}
```

Programme principal

```
void main(void)
{
    init_gpt1(1250);

    T3R = 1; // Lancement du Timer T3

    // A partir de là la génération ndu signal carré
    //est automatique et autonome.

    do
    {
        //Boucle infinie où le microcontrôleur peut
        //faire autre chose.
    }
    while (1);
}
```

b) Générer une temporisation Tp_duree

Application directe

On veut générer une temporisation d'environ 1ms

Calculer les valeurs de T3I et du nombre d'impulsions à charger dans T3 et T2

On reprend la formule :

$$Tp_duree (ns) = 50ns * 8 * 2^{T3I} * nb_impulsions = 400ns * 2^{T3I} * nb_impulsions$$

$$\text{Donc } 1ms = 1000000ns = 400ns * 2^{T3I} * nb_impulsions$$

$$\text{soit } 2^{T3I} * nb_impulsions = 2500$$

On dresse la table T3I nb_impulsions

Table

T3I	0	1	2	3	4	5	6	7
Nb impulsions	2500	1250	624					

On prend T3I = 0 et nb_impulsions = 2500

Programme d'application

On veut utiliser cette temporisation pour :

- faire clignoter une led branchée sur le bit 0 du port P8 au rythme de la seconde
- pour réaliser un chronomètre de 00,00 à 99,99 s qui a une précision de 10ms, soit au centième de seconde. 3 poussoirs, start stop et reset actifs à 0 branchés respectivement sur les bits 0, 1 et 2 du port P7 permettent :

Start de lancer le chronométrage

Stop d'arrêter le chronométrage

Reset de mettre le chronomètre à 0

Pour traiter ce problème, nous pouvons constater que le microcontrôleur va faire plusieurs tâches :

- Compter le temps pour le clignotement de la led (variable unsigned int tp_cligno)
- compter le temps pour le chronométrage (variables unsigned int tp_10ms, tp_chrono)
- gérer les poussoirs

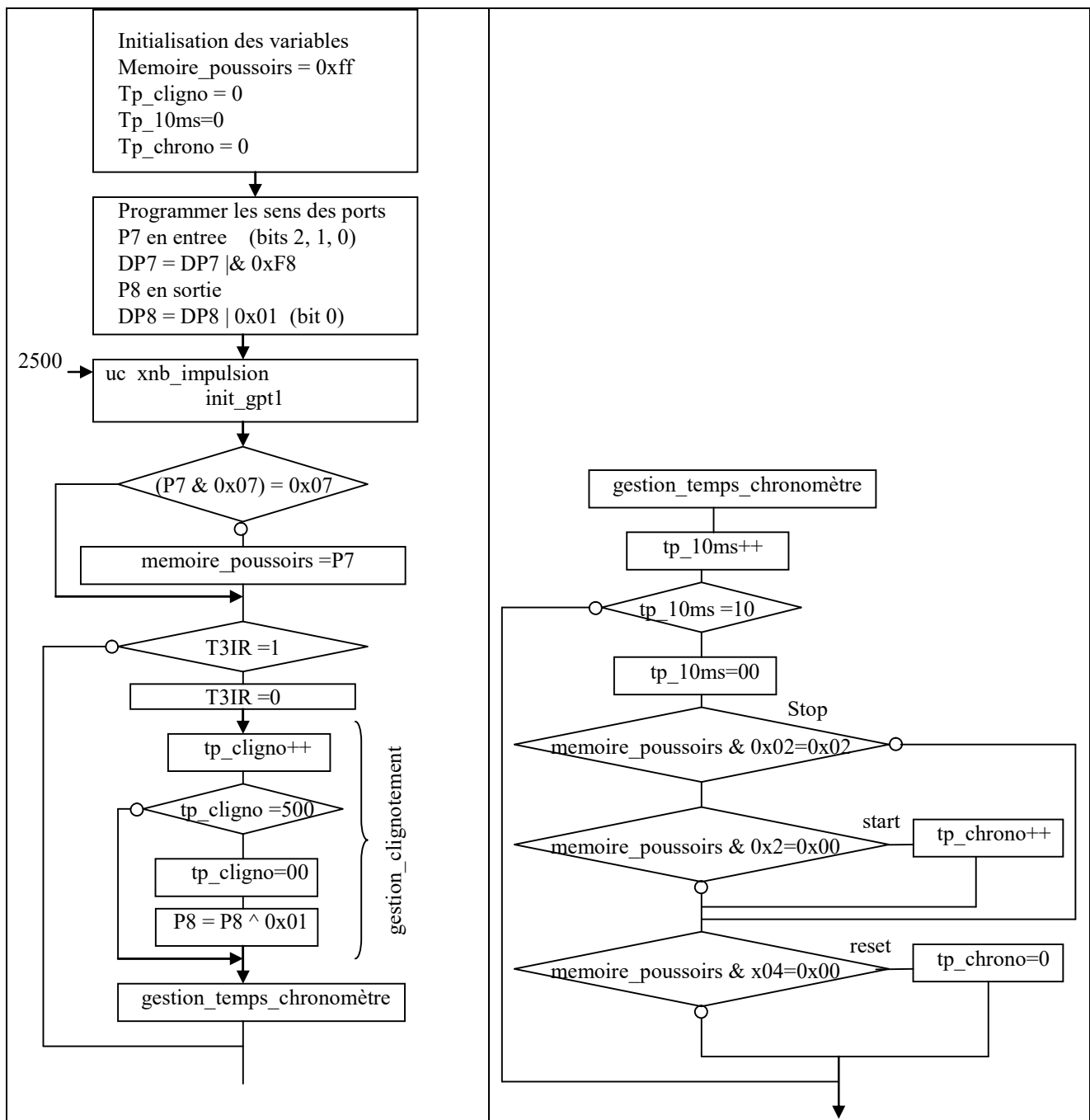
Nous partons d'une base de temps de 1ms

Donc tp_cligno va compter modulo 500 pour avoir 500ms

tp_10ms va compter modulo 10 pour avoir 10ms

Toutes les 10ms on doit mettre à jour tp_chrono si on est dans

Pour la gestion des poussoirs, nous allons définir une variable memoire_poussoirs qui est mis à jour si l'un des poussoirs est appuyé. On admettra que reset est plus prioritaire que stop lui-même plus prioritaire que start.



5-2 Le Groupe GPT2

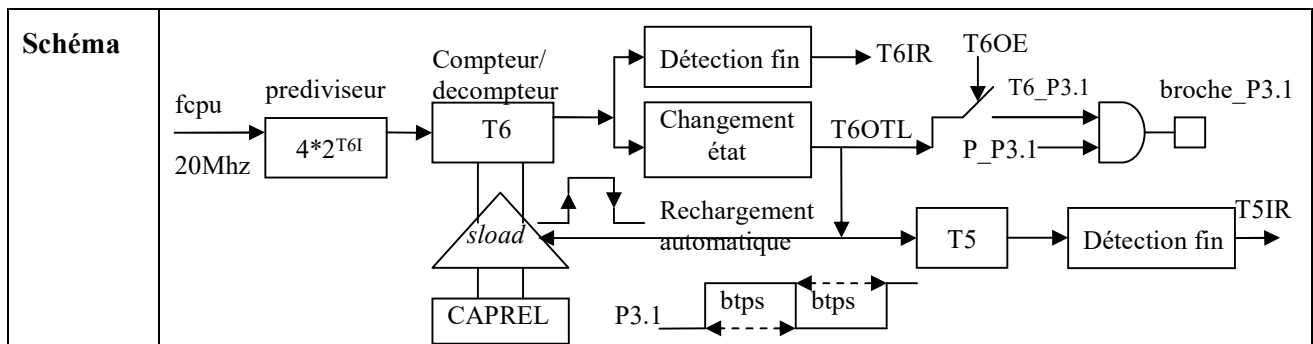
Ce groupe comporte 2 timers T5 et T6 avec comme timer principal T6. Il dispose d'un registre de rechargement automatique, CAPREL, pour le timer principal. Ce registre joue le même rôle que T2 dans la configuration que nous avons choisie et traitée.

Chaque timer dispose :

- d'un registre de contrôle-commande, appelé **registre de contrôle** et noté **TxCON** (x=5 ou 6)
- d'un registre de comptage/décomptage, qui est le **compteur/décompteur**, et noté **Tx** (x=5 ou 6)

Le fonctionnement de ce groupe est semblable à celui de GPT1. Toute l'explication que nous avons donnée pour le timer T3 s'applique au timer T6 avec comme registre de rechargement, CAPREL, qui remplace T2.

Nous avons vu que le groupe GPT1 permet de réaliser une temporisation de moyenne durée. Nous allons exploiter le groupe GPT2 pour réaliser une temporisation de longue durée.



a) Description du fonctionnement

T6 fonctionne en Timer et T5 fonctionne en compteur d'événements (fronts de T6OTL)

T6I, T6OTL, T6OE, T6IR et T5IR sont des bits que nous décrirons dans la suite

- le prédiviseur est un décompteur de modulo programmable $4 * 2^{T6I}$
- le compteur/décompteur est binaire sur 16bits (T6)
- le registre CAPREL (16bits) est le registre de rechargement automatique de T6 à la fin de chaque comptage-décomptage
- le timer T5 (16bits) est utilisé comme compteur auxiliaire en cascade après T6 (T5 est commandé par les fronts de T6OTL)
- la détection de fin (T6IR resp T5IR) :
 - ☞ de comptage se fait lorsque le compteur passe de 0xFFFF à 0x0000 $\Rightarrow T3IR = 1$
 - ☞ de décomptage se fait lorsque le décompteur passe de 0x0000 à 0xFFFF $\Rightarrow T3IR = 1$

☞ Remarque importante :

C'est la responsabilité du programmeur de mettre ce bit à 0 pour une prochaine détection de fin

- à chaque fin de comptage ou de décomptage de T6
 - ☞ le signal T6OTL change d'état
 - ☞ le contenu du registre CAPREL est rechargé dans celui du registre T6
 - ☞ le Timer T5 est mis à jour (incréméntation ou décrémentation):
- l'état de T6OTL (T6_P3.1) peut être recopié (si T6OE=1) sur la broche d'entrée-sortie P3.1 si P_P3.1=1
P_P3.1 représente le bit 1 registre du port P3

Remarque importante :

Dans cette configuration, le groupe GPT6 permet de réaliser :

- ☞ sur T6IR, une temporisation de moyenne durée variant de $4 \cdot T_{cpu}$ à $4 \cdot 2^{T3I} \cdot 2^{16} \cdot T_{cpu}$ soit avec $T6I = 7$, $F_{cpu} = 20\text{Mhz}$ donc $T_{cpu} = 50\text{ns}$, on obtient une durée de 200ns à environ $1677721600\text{ns} \approx 17\text{s}$
- ☞ la génération d'un signal carré sur T6OTL qui peut être recopié sur la broche d'entrée-sortie P3.1
- ☞ sur T5IR, une temporisation de longue durée variant de 200ns jusqu'à $1677721600\text{ns} \cdot 2^{16} \approx 30,5\text{h}$

b) Programmation

Pour programmer les Timers, nous allons utiliser les registres de contrôle (T6CON et T5CON) et les registres de comptage-décomptage (T6 et T5)

b-1) Les registres de contrôle

Puisque nous avons réduit les modes de fonctionnement, nous allons simplifier la programmation.

Le registre T6CON pour le timer T6

Le registre de contrôle T6CON est organisé en champs de la façon suivante :

T6CON

Numéro bit	15	14 à 11				10	9	8	7	6	5	4	3	2	1	0
Nom signal	T6SR					T6OTL	T6OE	T6UDE	T6UD	T6R	T6M			T6I		
Valeur imposée		0	0	0	0	1	1	0	1		0	0	0			

Les bits **T6OTL**, **T6OE**, **T6UD**, **T6SR** et **T6R** sont connus par le compilateur du langage C du C167.

T6M permet de choisir le mode de fonctionnement du Timer T6. Nous allons le fixer à 000 pour indiquer que T6 fonctionne en mode Timer (comptage du temps)

T6I permet de définir le modulo du prédiviseur lorsque le T6 fonctionne en Timer.

Ce modulo vaut $4 \cdot 2^{T3I}$ T6I est sur 3 bits et peut donc prendre les valeurs de 0 à 7.

Ce qui conduit à des prédiviseurs de $4 \cdot 2^0 = 4$ ($T6I = 0$) à $4 \cdot 2^7 = 512$ ($T6I = 7$)

T6I n'est pas connu par le compilateur, il faut trouver un mécanisme pour écrire cette valeur à la bonne position dans le registre T6CON

T6UDE ne sera pas utilisé. Nous le fixons à 0

T6OE est l'autorisation de la copie de T6OTL sur la broche d'entrée-sortie P3.1.

Nous le fixons à 1 pour permettre la génération de signal carré sur P3.1

T6OTL est le signal qui change d'état à chaque fin de comptage ou décomptage. Nous l'initialiserons à 0.

T6UD permet de choisir entre le comptage ($T6UD = 0$) et le décomptage ($T6UD = 1$)

T6R permet d'autoriser ($T6R = 1$) ou d'arrêter ($T6R = 0$) le comptage/décomptage.

T6SR permet le rechargement automatique ($T6SR = 1$) de CAPREL dans T6. Nous le fixerons à 1.

A l'initialisation, $T6CON = 0x0000 \Rightarrow$ tous les bits et les champs sont à 0

Le registre T5CON pour le timer T5 (utilisé en registre de rechargement)

Le registre de contrôle T5CON est organisé en champs de la façon suivante :

T5CON

Numéro bit	15 à 9							8	7	6	5	4	3	2	1	0
Nom signal								T5UDE	T5UD	T5R	T5M			T5I		
Valeur imposée	0	0	0	0	0	0	0	0	0		1	0	0			

Les signaux binaires **T5UD** et **T5R** sont connus par le compilateur du langage C du C167.

T5M permet de choisir le mode de fonctionnement du Timer T3. Nous allons le fixer à 001 pour indiquer que T5 fonctionne en mode compteur d'événements de T6OTL (fronts de T6OTL)

T5I permet de définir quand le rechargement de T3 doit se faire.

Comme nous voulons générer un signal carré sur P3.3, nous allons considérer que le rechargement se fera à toutes les fins de comptage ou décomptage c'est-à-dire sur les fronts montants et descendants de T6OTL. Pour cela, on doit programmer T5I = 111

T5UDE ne sera pas utilisé. Nous le fixons à 0

T5UD permet de choisir entre le comptage (T5UD = 0) et le décomptage (T5UD=1)

T5R permet d'autoriser (T5R=1) ou d'arrêter (T5R=0) le comptage/décomptage

A l'initialisation, T5CON = 0x0000 $\Rightarrow$ tous les bits et les champs sont à 0

b-2) Les registres de comptage-décomptage

Les registres de comptage-décomptage sont des compteurs-décompteurs programmables dont on peut définir la valeur initiale:

- soit en écrivant directement dans le registre
- soit par le registre de rechargement automatique

Les registre de comptage-décomptage pour le groupe GPT2 sont T6 et T5.

Rappel :

La fin du comptage est indiquée par T6IR =1 lorsque le compteur T6 passe de valeur_max à 0

La fin du décomptage est indiquée par T6IR =1 lorsque le décompteur T6 passe de 0 à valeur_max

La fin du comptage est indiquée par T5IR =1 lorsque le compteur T5 passe de valeur_max à 0

La fin du décomptage est indiquée par T5IR =1 lorsque le décompteur T5 passe de 0 à valeur_max

Calcul des valeurs à charger dans T5 et T6

Nous avons montré comment déterminer les valeurs initiales à charger dans les compteurs-décompteurs.

La méthode est la même.

Rappel

Si on veut compter x impulsions il faut charger $-x$

Si on veut décompter x impulsions il faut charger $x-1$

b-3) Détection de fin de comptage-decomptage

La détection de fin de comptage ou décomptage se fait par

- le bit T6IR pour le timer T6 mis à 1
- le bit T5IR pour le timer T5 mis à 1

Important:

Il est de la responsabilité du programmeur de remettre ces bits (T6IR et T5IR) à 0 pour permettre la prochaine détection de fin de comptage-décomptage.

b-4) Conclusion

Pour programmer les timers de groupe GPT2, on a deux étapes :

- la préparation des valeurs d'initialisation à programmer dans les registres, **T6CON**, **T5CON**, **T6** et **T5**
- la programmation effective qui comporte 2 parties :
 - ☞ les fonctions d'initialisation
 - ☞ le programme principal qui exploite le timer

a) Préparation des valeurs d'initialisation.

T6 est utilisé comme une base de temps et T5 est utilisé comme compteur-décompteur

On doit donc définir la durée, *duree_btps*, de la base de temps. On peut considérer que T6 génère un signal carré dont les deux fronts (montants et descendants) commandent le compteur-décompteur T5 pour obtenir une temporisation sur T5IR de durée, *Tp_duree*.

On calcule les paramètres à programmer dans les timers en particulier le nombre d'impulsions permettant de déterminer les valeurs à programmer dans T6 et T5

Soit *duree_btps*, la durée de la base de temps que T6 doit générer

Soit *Tp_duree*, la durée de la temporisation que T5 doit générer

Soit *nb_impulsions*, le nombre d'impulsions correspondant à la durée, *Tp_duree*.

Soit *nb_impulsions_T6* le nombre d'impulsions que T6 comptera ou décomptera

Soit *nb_impulsions_T5* le nombre d'impulsions que T5 comptera ou décomptera

$$nb_impulsions = nb_impulsions_T6 * nb_impulsions_T5$$

On a la relation

$$Tp_duree = Tcpu * modulo_prediviseur_T6 * nb_impulsions_T6 * nb_impulsions_T5$$

$$Tcpu = 50ns \quad \text{pour } fcpu = 20Mhz \quad \text{Modulo prédiviseur_T6} = 4 * 2^{T6I}$$

$$Tp_duree = 50ns * 4 * 2^{T6I} * nb_impulsions_T6 * nb_impulsions_T5$$

$$\text{Soit } 2^{T6I} * nb_impulsions_T6 * nb_impulsions_T5 = Tp_duree(ns)/200$$

On doit donc choisir le triplet **T6I**, **nb_impulsions_T6** et **nb_impulsions_T5** sachant que :

$$\text{☞ } T6I \leq 7$$

$$\text{☞ } nb_impulsions_T6 \leq 65536 \quad (\text{attention } 65536 \text{ conduit à charger } 0)$$

$$\text{☞ } nb_impulsions_T5 \leq 65536 \quad (\text{attention } 65536 \text{ conduit à charger } 0)$$

T6I	nb_impulsions_T6
0	
1	
2	
3	
4	
5	
6	
7	

Calcul de T6I, nb_impulsions_T6 et nb_impulsions_T5

Trouver le triplet constitué du plus petit T6I, *nb_impulsions_T6* et *nb_impulsions_T5* qui vérifie la relation.

Nous conseillons de procéder par étapes :

a-1) Calcul de T6I et nb_impulsions_T6

1) Fixer la base de temps, *duree_btps*

2) Dresser une table pour calculer T6I et *nb_impulsions_T6* en utilisant la relation $2^{T6I} * nb_impulsions_T6 = duree_btps(ns)/200$

Pour chaque valeur de T6I, on calcule *nb_impulsions_T6*

3) On identifie dans la table les couples T6I et *nb_impulsions_T6* avec *nb_impulsions_T6* ≤ 65536

4) On choisit parmi les couples identifiés celui qui possède le plus petit T6I

b) Calcul de nb_impulsions_T5

Connaissant *duree_btps* (donc T6I et *nb_impulsions_T6*) on peut en déduire *nb_impulsions_T5* en utilisant la relation $Tp_duree = duree_btps * nb_impulsions_T5$

$$\text{Soit } nb_impulsion_T5 = Tp_duree/duree_btps$$

Remarque :

On aurait pu faire $Tp_duree = 200ns * 2^{T6I} * nb_impulsions$

On en déduit $2^{T6I} * nb_impulsions = Tp_duree(ns)/200$

- 1- Dresser une table pour calculer T6I et nb_impulsions en utilisant la relation $2^{T6I} * nb_impulsions = Tp_duree(ns)/200$

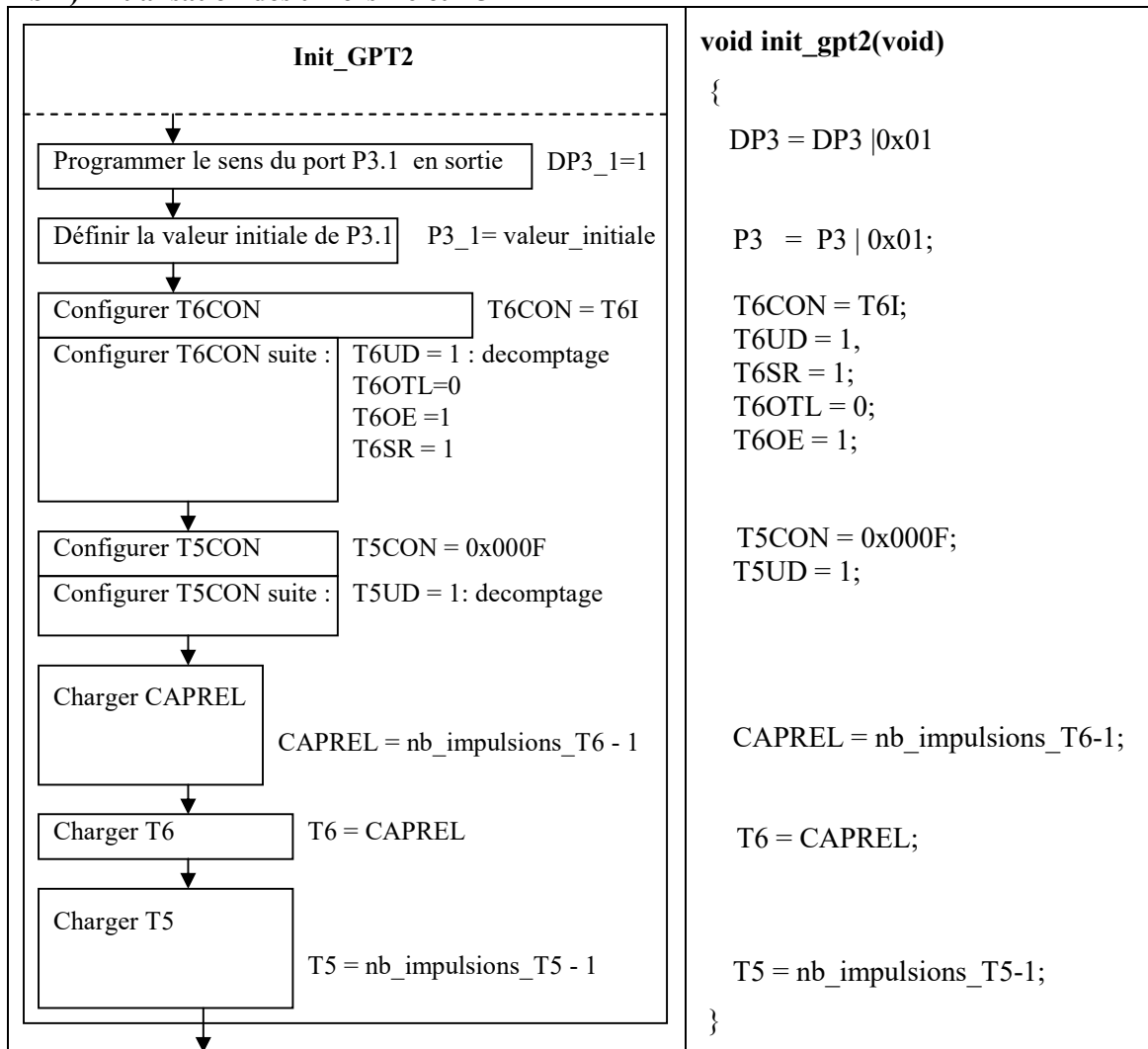
Pour chaque valeur de T6I, on calcule nb_impulsions

- 2- On identifie dans la table les couples T6I et nb_impulsions avec $nb_impulsions \leq 2^{16} * 2^{16}$

- 3- On choisit parmi les couples identifiés celui qui possède le plus petit T6I

- 4- On sait que $nb_impulsions = nb_impulsions_T6 * nb_impulsions_T5$ et nb_impulsions provient de 3)
On en déduit le couple nb_impulsions_T6, nb_impulsions_T5
Plusieurs solutions sont possibles.

b) on applique l'organigramme suivant en utilisant les valeurs calculées en a)

b-1) Initialisation des timers T6 et T5

b-2) Programme principal

